Code and Branch Coverage Analysis

Cloud Storage Tier Application | Formal Laboratory Report

Name: Dabbugunta Venya Anand
Date: 6 November 2026
File: cloud_storage_tier.py

1. Objective and scope

The objective of this laboratory exercise was to assess the supplied application, identify untested statements and decisions, and develop separate test suites achieving at least 90% code and branch coverage. Testing covers tenant storage billing and replication-latency monitoring. The application source was left unchanged.

2. Method and baseline

The supplied starter test was executed first. Additional tests were then developed in two stages: a statement-focused suite followed by a decision- and boundary-focused suite. Tests were executed using Python, pytest 9.2.0 and coverage.py 7.17.2. Coverage measurement was restricted to the application module.

The initial test passed, but executed only **26 of 57 statements (45.61%)** and **14 of 40 branch exits (35.00%)**. The branch-enabled tool reported a combined figure of 41.24%; this combined value is distinct from the separate statement and branch percentages.

3. Results summary

Code coverage suite: 7 tests passed; 57 of 57 statements executed; 0 missing statements; **100% statement coverage**.

Branch coverage suite: 41 tests passed; 57 of 57 statements executed; 40 of 40 branch exits covered; 0 partial branches; **100% branch coverage**.

4. Interpretation

Statement coverage measures whether executable statements run. Branch coverage additionally measures whether the alternative exits from decisions are exercised. Both final suites exceed the required 90% threshold. Complete branch coverage does not establish that every possible execution path or input combination is correct.

Code Coverage Analysis

Statement-focused tests and terminal evidence

5. Test suite and coverage approach

Test file: CodeCoverageTest_cloud_storage_tier.py
Seven tests exercise input validation, active and inactive tenants, supported and unsupported storage tiers, invalid and corrupted buckets, storage discounts, egress charges, archive reset and loop termination, backup support fees, billing outcomes, and replication-monitoring results.

The tests check returned values and errors as well as executing the relevant statements. The suite includes missing replication data, a sustained-latency alert, and a synchronized result. Measured coverage increased from 45.61% to 100%, with all 57 executable statements exercised.

6. Terminal output evidence

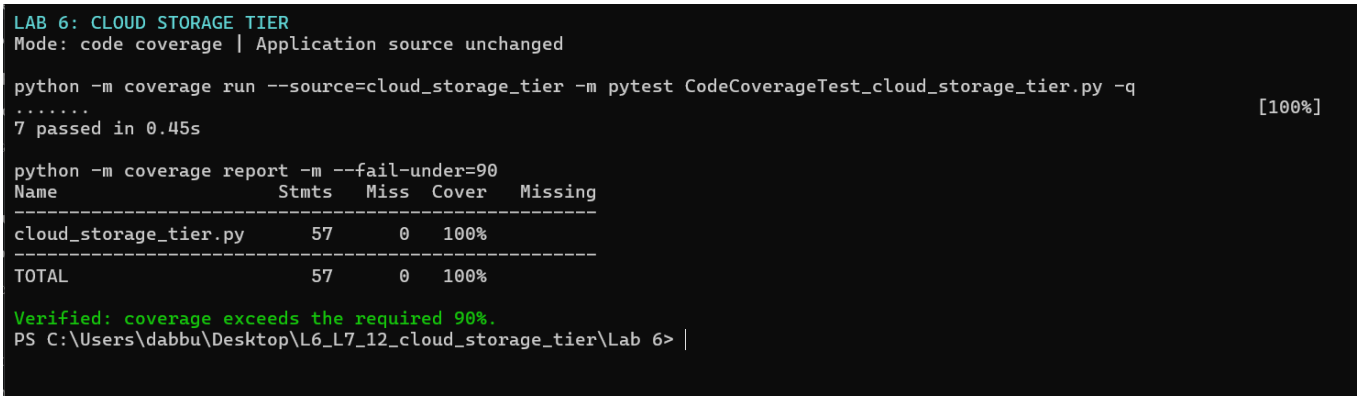


Figure 1. Actual terminal screenshot: 7 passing tests and 100% statement coverage. The coverage threshold check succeeds.

7. Reproduction procedure

From the Lab 6 directory, with pytest and coverage installed, execute:

```
python -m coverage run --source=cloud_storage_tier
-m pytest CodeCoverageTest_cloud_storage_tier.py -q

python -m coverage report -m --fail-under=90
```

The first command is displayed across two lines for readability and must be entered as one command.

Supporting evidence: evidence/code-tests.txt, evidence/code-coverage.txt and evidence/code-coverage.json retain the measured output and coverage data. The original starter test remains available separately.

Branch Coverage Analysis

Decision boundaries, terminal evidence and submission records

8. Test suite and boundary analysis

Test file: BranchCoverageTest_cloud_storage_tier.py
The 41-test suite covers valid and invalid inputs, tier alternatives, empty and nonempty loops, corruption handling, backup-fee decisions and billing-status precedence. Boundary cases include 500/501 GB storage, 100/101 GB egress, four/five audit flags and 200/201 ms latency. Replication tests check consecutive high readings, streak resets and isolation between nodes.

9. Terminal output evidence

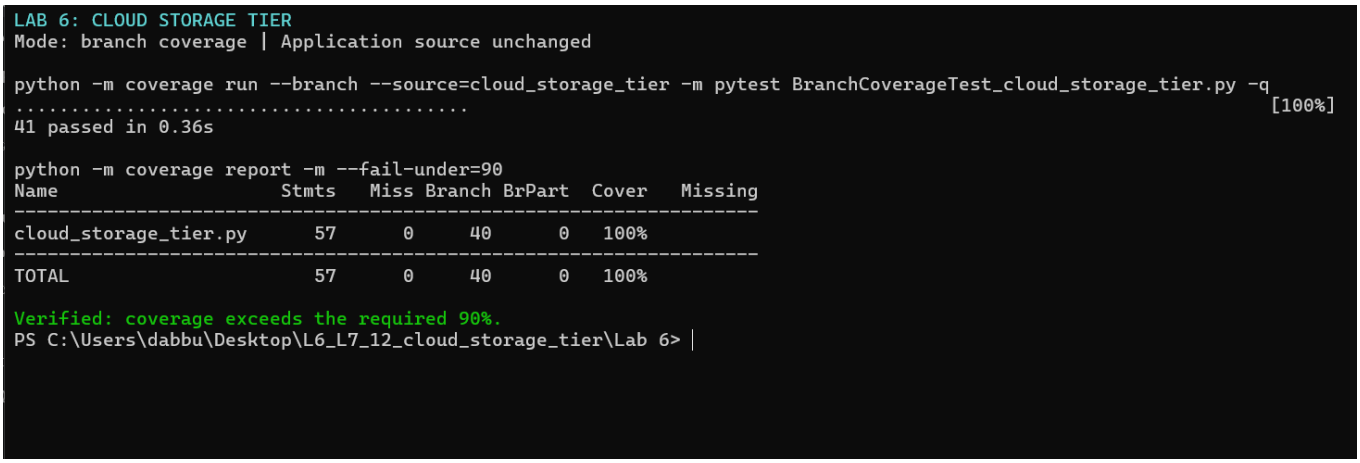


Figure 2. Actual terminal screenshot: 41 passing tests, 40 branch exits, zero partial branches and 100% coverage.

All 40 measured branch exits were exercised, compared with 14 in the baseline. The suite also verifies archive corruption resets accumulated storage charges and terminates bucket processing; any applicable backup fee is still evaluated afterward, matching the supplied implementation.

10. Reproduction and commit records

```
python -m coverage run --branch --source=cloud_storage_tier
-m pytest BranchCoverageTest_cloud_storage_tier.py -q
python -m coverage report -m --fail-under=90
```

Enter the first two displayed lines as one command. Detailed results are retained in evidence/branch-tests.txt, evidence/branch-coverage.txt and evidence/branch-coverage.json.

Separate implementation commits: 7ff3323 - code coverage tests and analysis; 2944cd8 - branch coverage tests and analysis.
Screenshot commits: e6a2867 - code coverage evidence; 585441f - branch coverage evidence. These records provide separate commits for the two required test suites.

Conclusion: Both coverage objectives were achieved without modifying the application source. The two test suites, separate commit records and terminal screenshots provide the required evidence for the laboratory submission.